



K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 3

Comprehensive Implementation of Page Replacement
Algorithms

Submitted by:

Name: Aditya Shibu

Roll Number: 2401201047

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

OS Lab Assignment 3 - Page Replacement Algorithms Report

□ Cover Section

- **Name:** Aditya Shibu
 - **Roll Number:** 2401201047
 - **Course:** BCA (AI & DS)
 - **University:** K.R. Mangalam University, School of Engineering & Technology
 - **Subject:** Operating System Lab (ENCA252)
 - **Submitted To:** Dr. Jyoti Yadav
 - **Date:** April 19, 2026
-

□ Objective

Implement and analyze five major page replacement algorithms: **FIFO**, **LRU**, **Optimal**, **MRU**, and **Second Chance (Clock)**. The goal is to simulate memory behavior and evaluate performance based on **page faults**.

Note

Sample used throughout: Reference String — 7 3 0 3 2 1 2 0 7 0 1 2 , Frames — 3

1. FIFO (First-In, First-Out)

How it works:

The oldest page in memory (the one that arrived first) is replaced when a new page needs to be loaded and all frames are full.

Logic:

A simple queue is maintained. When a page fault occurs and frames are full, the page at the front of the queue (oldest) is evicted and the new page is added to the rear.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☒ No
5	2	2	3	0	☒ Yes
6	1	2	1	0	☒ Yes
7	2	2	1	0	☒ No
8	0	2	1	0	☒ No
9	7	2	1	7	☒ Yes
10	0	0	1	7	☒ Yes
11	1	0	1	7	☒ No
12	2	0	2	7	☒ Yes

Total Page Faults: 8

2. LRU (Least Recently Used)

How it works:

The page that has not been used for the longest period of time is replaced.

Logic:

Usage history is tracked for each page in memory. On a page fault, the page with the oldest last-use timestamp is evicted. Exploits **temporal locality** — recently used pages are likely to be used again soon.



Tip

LRU generally performs close to Optimal and is widely used in practice for its balance of efficiency and implementability.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☒ No
5	2	2	3	0	☒ Yes
6	1	2	3	1	☒ Yes
7	2	2	3	1	☒ No
8	0	2	0	1	☒ Yes
9	7	2	0	7	☒ Yes
10	0	2	0	7	☒ No
11	1	1	0	7	☒ Yes
12	2	1	0	2	☒ Yes

Total Page Faults: 9

3. Optimal Page Replacement

How it works:

Replaces the page that will **not be used for the longest time in the future**.

Logic:

Looks ahead in the reference string to determine which page currently in memory will be needed farthest in the future (or not at all), and evicts that page. This is a theoretical algorithm — it cannot be implemented in real systems as it requires future knowledge.

Note

Optimal serves as a **benchmark** — no algorithm can produce fewer page faults than Optimal for the same input.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☒ No
5	2	7	3	2	☒ Yes
6	1	7	1	2	☒ Yes
7	2	7	1	2	☒ No
8	0	0	1	2	☒ Yes
9	7	7	1	2	☒ Yes
10	0	0	1	2	☒ Yes
11	1	0	1	2	☒ No
12	2	0	1	2	☒ No

Total Page Faults: 8

4. MRU (Most Recently Used)

How it works:

The **most recently used** page is replaced when a page fault occurs.

Logic:

Tracks which page was accessed last. On a fault, that most-recently-used page is evicted. This is the opposite of LRU and typically performs poorly for general workloads, but can be efficient for certain access patterns (e.g., sequential scans).



MRU can outperform LRU in specific scenarios such as cyclic sequential scans, where recently used pages are unlikely to be reused immediately.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☒ No
5	2	7	2	0	☒ Yes
6	1	7	1	0	☒ Yes
7	2	7	2	0	☒ Yes
8	0	7	2	0	☒ No
9	7	7	2	0	☒ No
10	0	0	2	0	☒ No
11	1	0	2	1	☒ Yes
12	2	0	2	1	☒ No

Total Page Faults: 7

5. Second Chance (Clock Algorithm)

How it works:

A modified version of FIFO. Each page has a **reference bit**. When a page is accessed, its bit is set to 1. On replacement, if the oldest page's bit is 1, it gets a "second chance" (bit reset to 0, moved to back of queue); if 0, it is replaced.

Logic:

Maintains a circular queue (clock). The "clock hand" sweeps through pages. A page with reference bit = 1 is spared once; on the next sweep if still unreferenced, it is evicted. Approximates LRU with lower overhead.

Note

The Second Chance algorithm is used in real OS implementations (e.g., Linux) as an approximation of LRU without the full overhead of tracking timestamps.

Step-by-Step Trace (*Bits shown in parentheses: 1 = referenced*)

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7(1)	-	-	☒ Yes
2	3	7(1)	3(1)	-	☒ Yes
3	0	7(1)	3(1)	0(1)	☒ Yes
4	3	7(1)	3(1)	0(1)	☒ No
5	2	2(1)	3(1)	0(1)	☒ Yes
6	1	2(1)	1(1)	0(1)	☒ Yes
7	2	2(1)	1(1)	0(1)	☒ No
8	0	2(1)	1(1)	0(1)	☒ No
9	7	2(1)	1(1)	7(1)	☒ Yes
10	0	0(1)	1(1)	7(1)	☒ Yes
11	1	0(1)	1(1)	7(1)	☒ No
12	2	0(1)	2(1)	7(1)	☒ Yes

Total Page Faults: 8

III Performance Comparison

Algorithm	Total Page Faults	Efficiency Rank
MRU	7	1st (Best for this string)
FIFO	8	2nd
Optimal	8	2nd (Theoretical minimum)
Second Chance	8	2nd
LRU	9	5th (Worst)

☑ Conclusion

- **Best Performer: Optimal** is always the theoretical best, but for this specific reference string, **MRU** coincidentally tied or beat others. Optimal remains the true benchmark (8 faults) that no real algorithm can beat.
- **Worst Performer: LRU** had the highest fault count (9) for this particular string, demonstrating that performance is highly input-dependent.
- **Observations: FIFO** and **Second Chance** tied with Optimal at 8 faults, showing that simpler algorithms can match complex ones on certain workloads. **Second Chance** is generally preferred over plain FIFO in real systems as it approximates LRU with minimal extra overhead. In production OS design, **LRU approximations** (like Second Chance/Clock) are the most common choice due to their balance of practicality and performance.